

Cours PS - Scripting et Automatisation (Windows)

Suivi documentaire

Éditeur	Version	Date	Modification
Thomas SCULFORT	1.0	28/03/2025	Version Initiale
Thomas SCULFORT	1.1	11/05/2025	Ajout Header/Commentaires

1- Introduction aux Langages de Scripting.....	3
1.1 Historique de Powershell.....	3
1.2 Pourquoi automatiser ?.....	3
1.3 Qu'es-ce qu'un IDE/EDI, lequel utiliser ?.....	3
1.4 Utilisation de l'IDE/ISE.....	4
1.5 Politique exécution de scripts (différence exécution/enregistrement).....	4
1.7 En-tête de Script (script header ou comment-based help).....	7
2- Généralités sur PowerShell.....	8
2.1 Syntaxe de base et principaux concepts.....	8
Variables :.....	8
Affectation valeur à une variable.....	8
Affiche contenu d'une variable.....	8
Afficher le type d'une variable.....	8
Les variables d'environnement :.....	9
Portée des variables :.....	9
Conditions (Si).....	11
Comparaisons :.....	11
Opérateurs.....	11
Opérateurs d'égalité.....	11
Opérateurs de comparaison : supérieur ou inférieur.....	11
Opérateurs de comparaison : ressemblance.....	11
Opérateurs de comparaison : contient ou ne contient pas.....	11
Boucles :.....	12
Switch (Selon cas).....	12
For.....	12
ForEach.....	12
Fonctions :.....	13
Appel.....	13
Cmdlet de base :.....	13
2.2 Utilisation de l'environnement PowerShell ISE.....	14

3- Administration et Automatisation avec PowerShell avancée	15
3.1 Gestion des utilisateurs et des groupes (AD).....	15
3.2 Configuration des stratégies de sécurité (GPO).....	15
3.3 Automatisation des tâches d'audit	15
3.3.1 Vérification des comptes utilisateurs inactifs.....	15
3.3.2 Surveillance des fichiers journaux (logs).....	16
3.3.3 Vérification de l'intégrité des fichiers système.....	16
3.3.4 Évaluation des paramètres de sécurité	16
3.3.5 Contrôle des permissions d'accès aux ressources	16
3.3.6 Tuer/Killer le processus "msedge.exe"	17
3.4 Utilisation de PowerShell pour la gestion des fichiers logs.....	18

1- Introduction aux Langages de Scripting

Les langages de scripting jouent un rôle essentiel dans le développement logiciel et l'administration informatique. Contrairement aux langages compilés, les scripts sont généralement interprétés, ce qui les rend particulièrement adaptés pour automatiser des tâches, manipuler des fichiers, ou orchestrer des processus.

1.1 Historique de Powershell

Avant Powershell on utilisait MS-DOS (v1 en 1981, v6 en 1994) qui permet/permettait de réaliser des scripts.

Windows PowerShell (2006) est une suite logicielle développée par Microsoft qui intègre :

- une interface en ligne de commande,
- un langage de script nommé PowerShell
- un kit de développement.

Il est inclus dans Windows 7, Windows 8.1, Windows 10 et Windows 11 (y compris les versions grand public/home) et s'appuie sur le framework Microsoft .NET.

Petit historique :

Powershell version 1.0 => 2006 (sous Windows XP, Server 2003, Vista)

Powershell version 2.0 => 2009 (sous Windows 7, Server 2008 R2)

Powershell version 3.0 => 2012 (sous Windows 8, Server 2012)

Powershell version 4.0 et 5.0 => 2013-2016 (Windows 10 sort en RC 2015)

Powershell version 6.0 (core) => 2018 Multiplateforme (Windows, Linux, MacOS)

Powershell version 7.0 => Ajout .NET, Azure, Docker...

1.2 Pourquoi automatiser ?

Nous pouvons réaliser les tâches d'administration via une interface graphique mais lorsque l'on veut configurer un ensemble d'équipements, nous devons passer par des scripts.

L'automatisation est un processus visant à utiliser des outils ou des technologies pour accomplir des tâches répétitives ou complexes de manière efficace, avec un minimum d'intervention humaine. Elle est essentielle pour :

- Gagner du temps : Réduire les efforts manuels et accélérer l'exécution des tâches.
- Améliorer la précision : Minimiser les erreurs humaines dans des processus critiques.
- Augmenter la productivité : Libérer les ressources humaines pour des tâches plus stratégiques.
- Réduire les coûts : Optimiser l'utilisation des ressources et limiter les gaspillages.
- Assurer la scalabilité : Réaliser facilement des tâches sur une plus grande échelle ou de manière récurrente.

Un administrateur/informaticien est (désormais) obligé d'automatiser ses tâches.

1.3 Qu'est-ce qu'un IDE/EDI, lequel utiliser ?

IDE (Integrated Development Environment) ou EDI (Environnement de Développement Intégré) en français, désigne un logiciel regroupant des outils nécessaires aux développeurs informatiques pour la création d'applications.

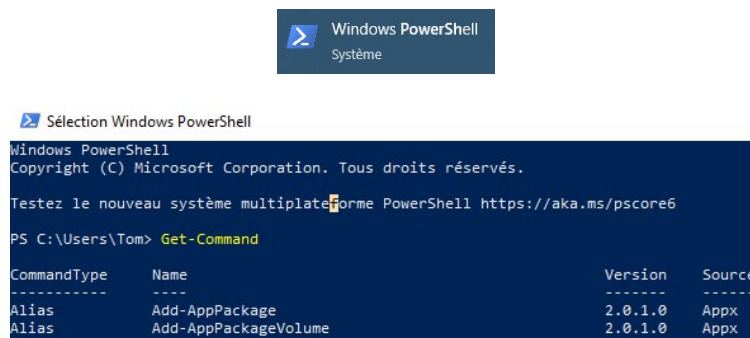
ISE (Integrated Scripting Environment) est un logiciel comparable à un IDE où l'on utilise notre langage de scripting.

Pour le langage powershell, Microsoft propose un IDE/ISE nommé « Powershell ISE » mais il existe d'autres logiciels comme « Microsoft Visual Studio Code » ou « Microsoft Visual Studio Community/Pro » ou « PowerShell Studio 2024 ».

1.4 Utilisation de l'IDE/ISE

En fonction des commandes utilisées vous devrez peut-être lancer/exécuter votre IDE/ISE avec les droits administrateur.

Pour exécuter des commandes on utilise simplement les consoles Powershell (64bits ou 32bits en fonction du processeur).



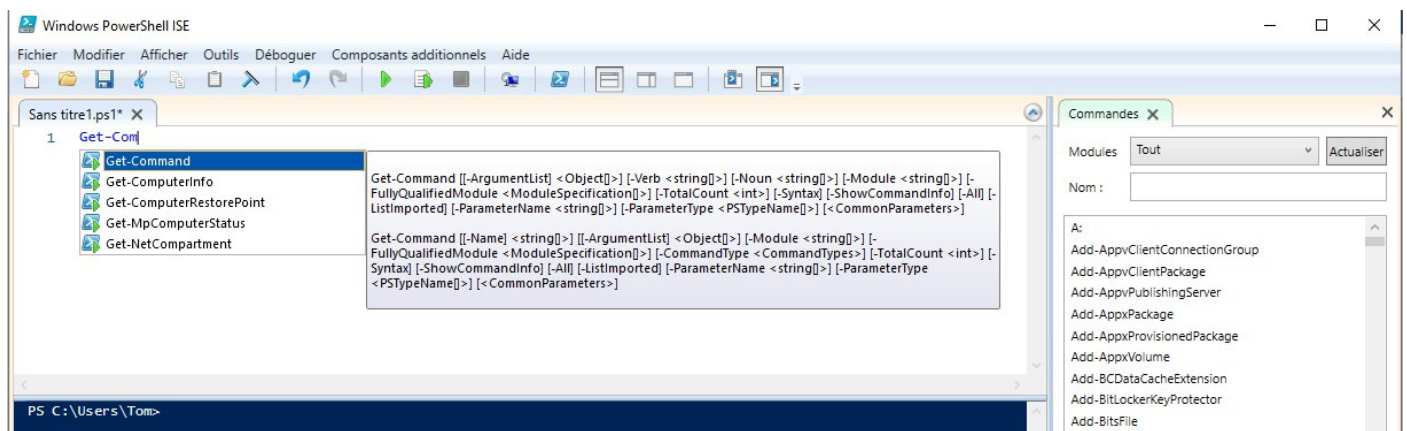
```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Testez le nouveau système multiplateforme PowerShell https://aka.ms/pscore6

PS C:\Users\Tom> Get-Command

CommandType      Name                                     Version      Source
-----
Alias             Add-AppPackage                         2.0.1.0      Appx
Alias             Add-AppPackageVolume                  2.0.1.0      Appx
```

Pour réaliser des Scripts on utilise un IDE, plus précisément un ISE, exemple avec Microsoft Powershell ISE (64bits ou 32bits en fonction du processeur).



1.5 Politique exécution de scripts (différence exécution/enregistrement)

La commande **Get-ExecutionPolicy** permet d'afficher la politique d'exécution des scripts de votre ordinateur.

La commande **Set-ExecutionPolicy** permet de modifier la politique d'exécution des scripts de votre ordinateur. Cette commande attend des paramètres, exemple :

AllSigned: Nécessite que tous les scripts et tous les fichiers de configuration soient signés par un éditeur approuvé, y compris les scripts écrits sur l'ordinateur local.

Bypass: rien n'est bloqué, et aucun avertissement, ni aucune invite ne s'affiche.

Default: Définit la stratégie d'exécution par défaut. Restricted pour les clients Windows ou RemoteSigned pour les serveurs Windows.

RemoteSigned: Exige que tous les scripts et tous les fichiers de configuration téléchargés à partir d'Internet soient signés par un éditeur approuvé. Stratégie d'exécution par défaut pour les ordinateurs Windows Server.

Restricted: Ne charge pas les fichiers de configuration ou n'exécute pas de scripts. Stratégie d'exécution par défaut pour les ordinateurs clients Windows.

Undefined: Aucune stratégie d'exécution n'est définie pour l'étendue. Supprime une stratégie d'exécution affectée d'une étendue qui n'est pas définie par une stratégie de groupe. Si la stratégie d'exécution dans toutes les étendues est Undefined, la stratégie d'exécution effective est Restricted.

Unrestricted : À compter de PowerShell 6.0, il s'agit de la stratégie d'exécution par défaut pour les ordinateurs non-Windows et ne peut pas être modifiée. charge tous les fichiers de configuration et exécute tous les scripts. Si vous exécutez un script non signé téléchargé à partir d'Internet, vous êtes invité à entrer l'autorisation avant son exécution.

1.6 Comment exécuter des scripts déjà réalisés

Depuis :

- une invite de commande Windows Powershell

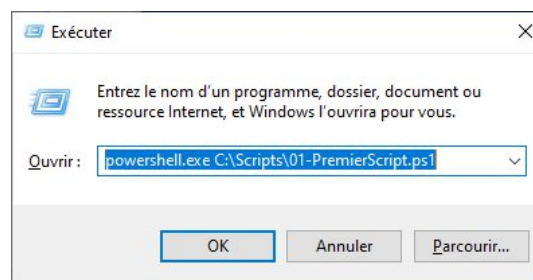
```
PS C:\Users\Tom> C:\Scripts\01-PremierScript.ps1
Ceci est mon premier script
Cliquez sur Entrée pour continuer...
PS C:\Users\Tom> powershell.exe C:\Scripts\01-PremierScript.ps1
Ceci est mon premier script
Cliquez sur Entrée pour continuer...
```

- une invite de commande « MS-DOS »

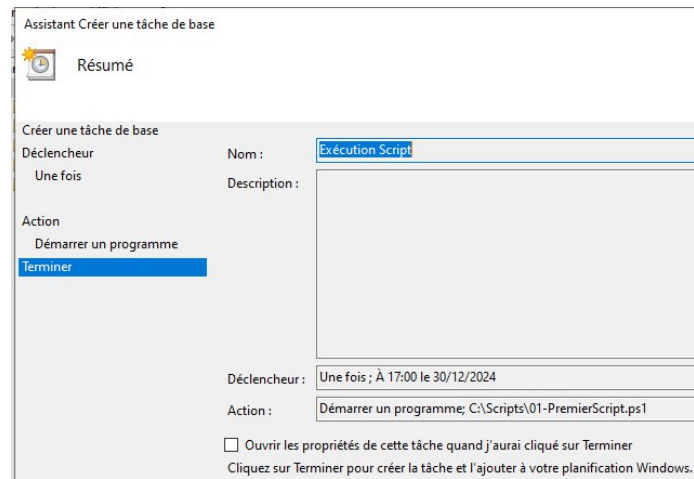
```
Ca: C:\Windows\system32\cmd.exe - powershell C:\Scripts\01-PremierScript.ps1

C:\Users\Tom>powershell C:\Scripts\01-PremierScript.ps1
Ceci est mon premier script
Cliquez sur Entrée pour continuer...
```

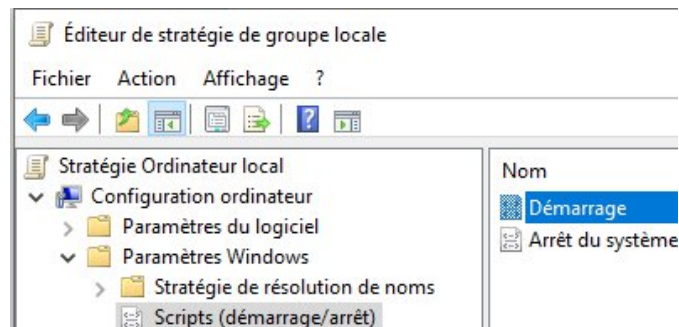
- une invite d'exécution de commande



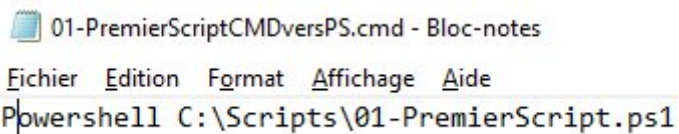
- une tâche planifiée (tâche de base)



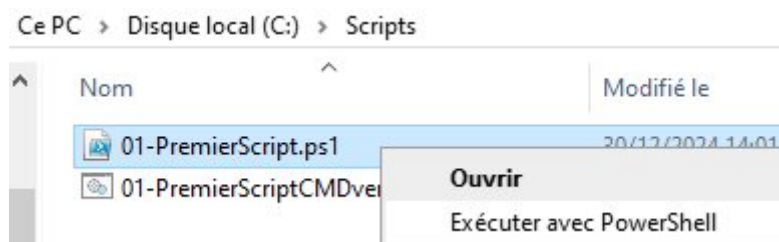
- les stratégies de groupe



- un fichier « MS-DOS » qui exécute votre Script Powershell



- Menu contextuel



1.7 En-tête de Script (script header ou comment-based help)

Un Script doit contenir un en-tête avec un minimum d'informations, sous GNU/Linux on appelle ceci un « Shebang », sous PS il faut donc garder cette bonne pratique et je vous propose l'en-tête suivant :

- Nom : Nom du fichier Script
- Auteur : Personnes ayant réalisées le script
- Date : date de création du script
- Version : Version actuelle du script
- VERSIONNING : Contient les dates et version du script (historique)
- Requires/Prérequis : indique quelle version de PS est nécessaire/environnement
-

```
# =====  
# NOM: Package Installation.ps1  
# AUTEUR: PRANDI et ATTARD  
# DATE: 12/10/2022  
#  
# VERSION Actuelle : 1.1  
# VERSIONNING :  
# 10/10/2022 : v1.0 Version initiale  
# 12/10/2022 : v1.1 Ajout de Zoom  
  
# COMMENTAIRES : Installation des trois applications : Thunderbird , Zoom, Chrome  
#  
# Requires : Version 2.0  
# =====  
<# Commentaire multi-lignes #>
```

Votre Script apr la suite doit inclure des commentaire, commenter grâce au « # » pour une ligne ou avec les balises ouvrantes et fermantes suivantes « <# » et « #> »

Pour des commentaires plus complet vous pouvez vous référer à la documentation Microsoft

https://learn.microsoft.com/fr-fr/powershell/module/microsoft.powershell.core/about/about_comment_based_help?view=powershell-7.6&utm_source=chatgpt.com

2- Généralités sur PowerShell

2.1 Syntaxe de base et principaux concepts

Cmdlets : Les commandes sont appelées des « cmdlets » (on prononce « commandelets »). Une commande commence souvent par un verbe (Get, Set, Add, Remove...)

Syntaxe : Les objets ne sont pas soumis à la casse.

Exemple : « **Get-Command** » est égale à « **gET-cOmMaNd** »

Propriétés : Les commandes ont des propriétés, il suffit de spécifier les paramètres à votre commande.

Exemple : « **Get-Command -Name A* -CommandType Cmdlet** »

Autocomplétion : La touche « Tab » permet de compléter le nom des objets, paramètres...

Pipeline (« | ») : Le « | » lui, sert à l'enchaînement de plusieurs commandes.

Export au format txt : On peut facilement exporter les résultats des cmdlets au format « .TXT »

Exemple : **Get-Command | Out-File -FilePath C:\Scripts\ListeCommandes.txt**

Export au format CSV : On peut facilement exporter les résultats des cmdlets au format « .CSV » (Comma separated Values).

Exemple : **Get-Command | Export-CSV C:\Scripts\ListeCommandes.csv**

Variables :

Affectation valeur à une variable

Exemple : **\$mavariabLe = "coucou"**

Affiche contenu d'une variable

Exemple : **write-output \$mavariabLe**

Afficher le type d'une variable

Exemple : **\$mavariabLe.GetType()**

Type de variable	Description
[String] System.String	Une chaîne de caractères, c'est-à-dire du texte. Exemple : \$Chaine = "Bonjour le monde"
[Integer] System.Int32	Un nombre entier. Exemple : \$Nombre = 7
[Double] System.Double	Un nombre décimal (64 bits) Exemple : \$NombreDecimal = 7.5

[Boolean] System.Boolean	Une valeur booléenne, c'est-à-dire vrai ou faux. Exemple : \$Booleen = \$true
[Array] System.Object	Un tableau, qui peut contenir plusieurs valeurs. Exemple 1 : \$Tableau = 1,2,3,4 Exemple 2 : \$Tableau = "bonjour","bonsoir","au revoir"
[HashTable] System.Object	Une table de hachage, soit une collection de paires clé/valeur. Exemple : \$HashTable = @{"1"="Un";"2"="Deux"}
[DateTime] System.DateTime	Une date et/ou un horaire. Exemple : \$Date = Get-Date

« *\$PSVersionTable* » : Variable contient des informations sur la version de PowerShell que vous utilisez, y compris la version principale, mineure et la révision.

« *\$PSHOME* » : Variable contient le chemin vers le répertoire où PowerShell est installé.

« *\$_* » : Variable automatique, elle représente l'objet actuel dans le pipeline. Lorsque nous utilisons une boucle, par exemple "ForEach-Object", nous utilisons cette variable. C'est également le cas lors de la création d'un filtre avec "Where-Object". À chaque fois, "\$_" représente l'objet actuel sur lequel la boucle est en train de travailler.

« *\$Error* » : La variable contient une liste des dernières erreurs qui se sont produites dans la session PowerShell. Elle est très pratique pour obtenir la liste des erreurs faisant suite à l'exécution d'un script, mais aussi pour la gestion des erreurs. Cette variable automatique correspond à un tableau (la première valeur d'un tableau est associée à l'index 0). La première erreur, c'est-à-dire la plus récente, est accessible via "\$Error[0]", la seconde via "\$Error[1]", etc.

La variable « *\$?* » : variable automatique contient le statut de la dernière opération (commande exécutée). Si la dernière commande s'est exécutée avec succès, "\$?" sera égale à "True", sinon, en cas d'échec, "\$?" sera "False". Cette variable automatique est de type booléen.

\$true), *\$false*, *\$null* : Ces variables représentent les valeurs booléennes vrai (\$true) et faux (\$false), ainsi que la valeur nulle (\$null).

Elles sont souvent utilisées dans les structures conditionnelles, que nous étudierons par la suite (if, else).

« *\$PSScriptRoot* » : Variable automatique qui contient le chemin du répertoire à partir duquel le script actuel est exécuté.

Les variables d'environnement :

Liste des variables de l'environnement : « Get-ChildItem Env: »

Utilisation « \$env:<nom de la variable> »

Portée des variables :

Global La variable est accessible partout dans la session PowerShell, y compris en dehors du script actuel. C'est la portée par défaut pour les variables créées dans une console PowerShell. Elle est également associée aux variables automatiques.

Local La variable est accessible uniquement à l'intérieur de la fonction ou du script où elle a été définie. C'est la portée attribuée par défaut à une variable dans un script (à moins qu'un nom de portée soit indiqué). Lorsque le script a terminé son exécution, la variable est inaccessible.

Script La variable est accessible partout dans le script actuel, mais pas en dehors de celui-ci.

\$Global:VariableGlobale

\$Local:VariableLocale

\$Script:VariableScript

Exemple : **`$global:Texte = "Hello World"`**

Concaténation : permet a des variable d'être « additionnée » si ce sont des entiers/nombre ou concaténées dans le cas de texte.

Exemple : `$var3 = $var1 + $var2`

Conditions (Si)

Exemple : `if ($MaVariable -eq $null) { Write-Host "La variable est nulle" }`

Comparaisons :

`Get-Command | Where-Object { $_.CommandType -eq "Cmdlet" }`

Opérateurs

Opérateurs d'égalité

"-eq" : Vérifier si une valeur est égale à une autre.

"-ne" : À l'inverse, pour vérifier l'inégalité et vérifier si deux valeurs sont bien différentes.

Opérateurs de comparaison : supérieur ou inférieur

opérateurs pour vérifier si une valeur est supérieure ou inférieure à une autre :

« -gt » : Greater than = Supérieur à...

« -ge » : Greater or equal = Supérieur ou égal à...

« -lt » : Less than = Inférieur à...

« -le » : Less or equal = Inférieur ou égal à...

Opérateurs de comparaison : ressemblance

opérateurs permettant de rechercher une correspondance plus ou moins proches entre deux valeurs

« -like » : Correspond à...

« -notlike » : Ne correspond pas à...

Exemple : `"Powershell" -like "?ower*"`

Ces opérateurs (-match & -notmatch) acceptent des modèles de correspondance que l'on appelle "pattern" et qui correspondent à des expressions régulières (RegEx). Bien que ce soit très puissant, ce n'est pas une notion évidente à appréhender.

-match : Correspond à l'expression régulière...

-notmatch : Ne correspond pas à l'expression régulière...

Opérateurs de comparaison : contient ou ne contient pas

Vérifier si une valeur est présente dans une collection, ou à l'inverse si la collection contient la valeur. Le terme collection faisant référence à un ensemble de valeurs

- « -contains » : La collection contient la valeur
- « -notcontains » : La collection ne contient pas la valeur
- « -in » : La valeur est présente dans la collection
- « -notin » : La valeur n'est pas présente dans la collection

Boucles :

Switch (Selon cas)

Évite l'utilisation de SI/IF imbriqués :

Exemple :

```
$NomLogiciel = Read-Host -Prompt "Quel logiciel souhaitez-vous lancer ?"
```

```
switch ($NomLogiciel)
{
    "notepad" { Start-Process notepad.exe }
    "powershell" { Start-Process powershell.exe }
    "calc" { Start-Process calc.exe }
    "regedit" { Start-Process regedit.exe }
    Default { "Désolé, je n'ai pas trouvé ce logiciel" }
}
```

For

L'objectif d'une boucle "For" est d'exécuter X fois un bloc d'instructions, où le nombre d'exécutions dépend de l'état de la condition testée.

```
For($i=0 ; $i -lt 10 ; $i++)
{
    Write-Output "La valeur est $($valeur[$i])"
}
```

ForEach

Une boucle "ForEach" va permettre de manipuler une collection de données dans son intégralité. Ceci est avantageux car la boucle "ForEach" va automatiquement traiter tous les éléments de notre collection, donc il n'y a pas besoin de connaître à l'avance le nombre qu'il y en a, comme c'est le cas avec une boucle "For".

Exemple :

```
Foreach ($element in Get-Service)
{
    "$($element.Name) dans l'état : $($element.Status) ($($element.StartType))"
}
```

Il existe d'autres boucles comme While, Do-While, Do-Until

Fonctions :

Une fonction est un bout de code, ou si vous préférez, un ensemble de commandes et d'instructions qui effectuent une tâche spécifique.

Création

```
function Write-Hello {  
    param([string]$Language = "FR")  
    Switch($Language)  
    {  
        "FR" { $Hello = "Bonjour" }  
        "EN" { $Hello = "Hello" }  
        "ES" { $Hello = "Hola" }  
        "IT" { $Hello = "Buongiorno" }  
        default { $Hello = "Bonjour" }  
    }  
    return $Hello  
}
```

Appel

```
Write-Hello("IT")
```

Cmdlet de base :

Clear : purge l'affichage de l'écran

Get-Command : Affiche la liste des commandes Powershell

Get-help « *command* » : Affiche l'aide sur une commande

Get-Process : Affiche les process du système

Get-Service : Afficher des services système

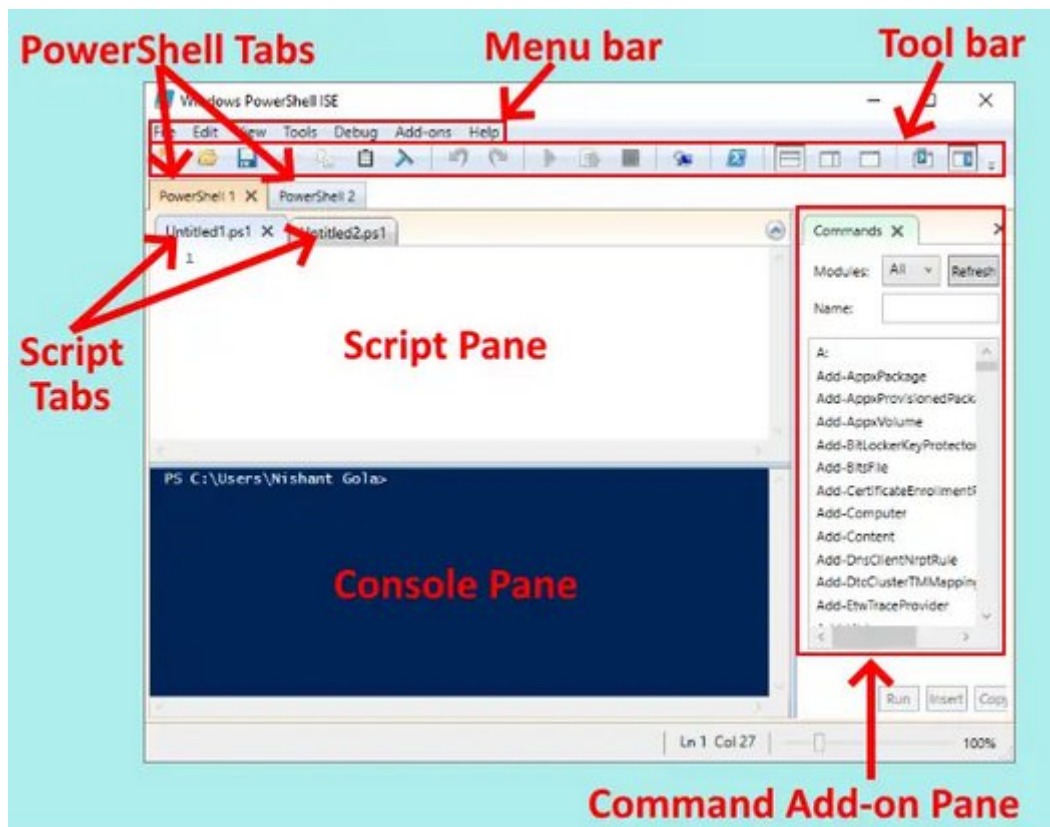
Start-Job : Démarre une tâche

Set-Location : Changer de répertoire (*Exemple : Set-Location C:\Scripts*)

Test-Path : Vérifier de l'existence de chemins d'accès (*Exemple : Test-Path C:\Scripts*)

Get-Childitem : Afficher le contenu d'un dossier (*Exemple : Get-Childitem -Path C:\Scripts*)

2.2 Utilisation de l'environnement PowerShell ISE



3- Administration et Automatisation avec PowerShell avancée

3.1 Gestion des utilisateurs et des groupes (AD)

Création utilisateur

`New-ADUser p.nom`

Modification utilisateur

`Get-ADUser`

Suppression utilisateur

`Remove-ADUser`

Activation/Désactivation Utilisateur

`Set-ADUser -Identity NomUtilisateur -Enabled:$false`

`Set-ADUser -Identity NomUtilisateur -Enabled:$true`

Création Groupe

`New-ADGroup`

Affectation utilisateurs dans un groupe

`Add-ADGroupMember -Identity MonGroupe -Members UtilisateurASuppr`

Suppression utilisateurs dans un groupe

`Remove-ADGroupMember -Identity MonGroupe -Members UtilisateurASuppr`

3.2 Configuration des stratégies de sécurité (GPO)

Par GPO il est possible de déployer des scripts Powerwhell.

Attention à la politique déployée (« **`Get-ExecutionPolicy`** » / « **`Set-ExecutionPolicy`** ») sur les postes.

Possibilité de contourner avec :

`Powershell.exe -ExecutionPolicy Bypass -File .\script.ps1`

ou

`Set-ExecutionPolicy RemoteSigned -Confirm:$false CurrentUser -Force`

3.3 Automatisation des tâches d'audit

Une tâche d'audit fait référence à une activité qui permet de surveiller, examiner et analyser les systèmes, les applications, ou les processus dans le but de vérifier la conformité, la sécurité, ou la performance d'un environnement informatique. Ces audits sont souvent réalisés pour s'assurer que les politiques de sécurité sont respectées, que les données sont protégées, et que les systèmes sont configurés correctement.

3.3.1 Vérification des comptes utilisateurs inactifs

Ce script vérifie les utilisateurs inactifs depuis plus de 90 jours :

```
# Récupérer tous les utilisateurs dans Active Directory
$users = Get-ADUser -Filter * -Property LastLogonDate
```

```
# Filtrer les utilisateurs inactifs depuis plus de 90 jours
$inactiveUsers = $users | Where-Object { $_.LastLogonDate -lt (Get-Date).AddDays(-90) }
```

```
# Afficher les utilisateurs inactifs
$inactiveUsers | Select-Object Name, SamAccountName, LastLogonDate
```

3.3.2 Surveillance des fichiers journaux (logs)

Ce script recherche dans le journal d'événements Windows les événements d'erreur récents dans les 24 dernières heures :

```
# Récupérer les erreurs du journal des événements
$errors = Get-WinEvent -LogName Application -FilterXPath "[System[Level=2]]" -StartTime (Get-Date).AddDays(-1)
```

```
# Afficher les erreurs récentes
$errors | Select-Object TimeCreated, Message, Id | Format-Table -AutoSize
```

3.3.3 Vérification de l'intégrité des fichiers système

Ce script vérifie l'intégrité des fichiers système en utilisant l'outil sfc (System File Checker) :

```
# Exécuter l'outil SFC pour vérifier l'intégrité des fichiers système
sfc /scannow
```

3.3.4 Évaluation des paramètres de sécurité

Ce script vérifie si le pare-feu Windows est activé sur l'ordinateur local :

```
# Vérifier l'état du pare-feu Windows
$firewallStatus = Get-NetFirewallProfile | Select-Object Name, Enabled

# Afficher l'état du pare-feu pour chaque profil
$firewallStatus
```

3.3.5 Contrôle des permissions d'accès aux ressources

Ce script vérifie les permissions sur un dossier spécifique et affiche les utilisateurs ayant un accès à ce dossier :

```
# Spécifier le chemin du dossier
$folderPath = "C:\Dossier\Important"

# Récupérer les ACL (listes de contrôle d'accès) du dossier
$acl = Get-Acl -Path $folderPath

# Afficher les utilisateurs et leurs permissions
```


`$acl.Access | Select-Object IdentityReference, FileSystemRights`

3.3.6 Tuer/Killer le processus "msedge.exe"

Ce script PowerShell pour rechercher le processus msedge.exe (Microsoft Edge) et le terminer (ou le "killer" comme tu l'as dit) si nécessaire :

`# Rechercher le processus msedge.exe`

`$process = Get-Process -Name "msedge" -ErrorAction SilentlyContinue`

`# Vérifier si le processus est en cours d'exécution`

`if ($process) {`

`# Terminer le processus`

`$process | ForEach-Object {`

`Write-Host "Terminer le processus msedge.exe avec PID: $($_.Id)"`

`Stop-Process -Id $_.Id -Force`

`}`

`} else {`

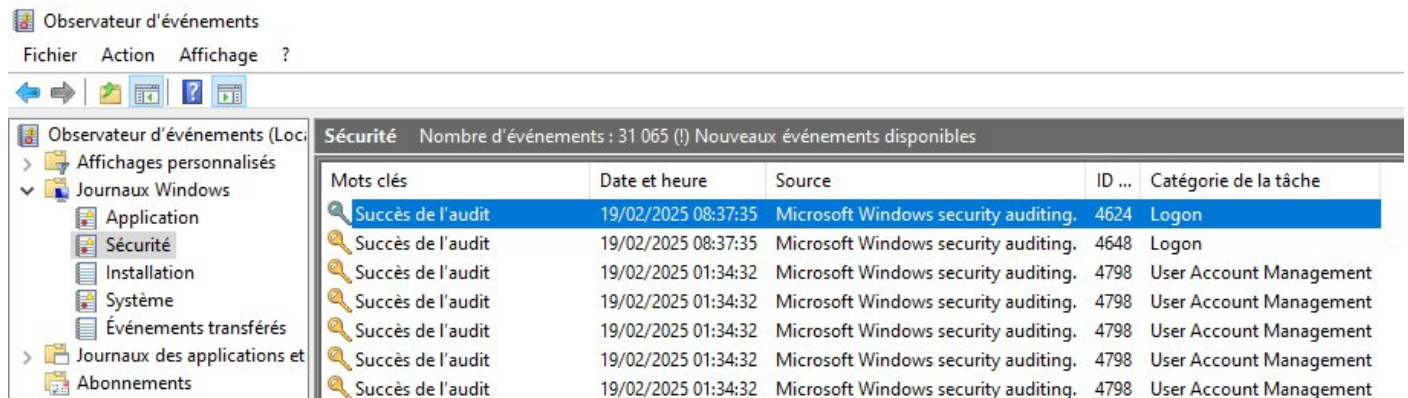
`Write-Host "Le processus msedge.exe n'est pas en cours d'exécution."`

`}`

3.4 Utilisation de PowerShell pour la gestion des fichiers logs

Il est possible de questionner/auditer les fichiers journaux de Microsoft via :

- L'interface graphique



- Powershell

Exemple, ce Script permet d'afficher les 10 dernières entrées du journal de Sécurité

```
Get-WinEvent -LogName Security | Select-Object -First 10
```

Plus on rajoute de critère, plus le résultat est long (**voir interminable**) à obtenir.

Exemple, récupération des connexions utilisateur pour une date donnée (10 minutes)

```
# Définir les dates de début et de fin pour filtrer
```

```
$startDate = Get-Date -Year 2025 -Month 2 -Day 18
```

```
$endDate = Get-Date -Year 2025 -Month 2 -Day 19
```

```
# Définir l'ID d'événement pour les connexions réussies (4624)
```

```
$eventIds = @(4624)
```

```
# Récupérer les événements de sécurité filtrés par date et ID d'événement
```

```
$logonEvents = Get-WinEvent -LogName Security | Where-Object {
```

```
    $_.TimeCreated -ge $startDate -and $_.TimeCreated -le $endDate -and $eventIds -contains $_.Id  
}
```

```
# Vérifier si des événements ont été trouvés
```

```
if ($logonEvents.Count -eq 0) {
```

```
    Write-Host "Aucun événement trouvé avec les critères spécifiés."
```

```
} else {
```

```
    # Extraire et formater les informations pertinentes pour chaque événement
```

```
$logonDetails = $logonEvents | ForEach-Object {
```

```
    $event = $_
```

```
# Récupérer les propriétés structurées des événements
```

```
$properties = @{
```

```
    EventId = $event.Id
```

```

    DateTime = $event.TimeCreated.ToString("yyyy-MM-dd HH:mm:ss")
    UserName = ""
    LogonType = ""
    EventMessage = $event.Message
}

# Si c'est l'événement 4624 (connexion réussie), nous extrayons le nom du compte et d'autres
détails
if ($event.Id -eq 4624) {
    # Utilisation des propriétés de l'événement pour extraire des informations structurées
    $properties.UserName = $event.Properties[5].Value # Account Name
    $properties.LogonType = $event.Properties[8].Value # Logon Type
}

# Créer un objet personnalisé avec toutes les informations extraites
[PSCustomObject]$properties
}

# Afficher les résultats sous forme de tableau
$logonDetails | Format-Table -AutoSize
}

```

Sources :

https://fr.wikipedia.org/wiki/Windows_PowerShell

<https://www.thewindowsclub.com/how-to-install-and-use-windows-powershell-ise-beginners-tutorial>

<https://www.malekal.com/commandes-powershell-base-apprendre-debuter/>

<https://www.it-connect.fr/chapitres/powershell-utilisation-du-pipeline/>

<https://www.it-connect.fr/modules/les-variables-et-les-operateurs-avec-powershell/>

<https://www.it-connect.fr/powershell-et-les-regex-expressions-regulieres/>

<https://learn.microsoft.com/fr-fr/powershell/module/microsoft.powershell.security/set-executionpolicy?view=powershell-7.4>

<https://www.it-connect.fr/chapitres/powershell-configurer-strategie-execution-des-scripts-ps1/>